

Benchmarking

Testing, evaluation, and benchmarking

Testing, Evaluation, and Benchmarking

Assuming you've either *created* a component in house or outsourced it, *evaluating* if that component actually functions as intended is still a required step

The 3 terms here mean

- Testing - Ensuring base functionality
- Evaluation - Ensuring functional requirements are met
- Benchmarking - Ensuring non functional requirements are met

Testing

Testing, in this context, ensures that the component built works as intended without bugs or errors

This is primarily done through *automated tests* and *user run throughs*.

Other techniques include unit testing, and integration testing

Unit Testing

Testing the smallest functional units of code to ensure internal logic is correct.

These should be fast and run automatically every time the code changes.

These include frameworks like pytest and unittest for *python*, jest for *javascript* and gtest for *c and c++*.

These run primarily on the concept of *assertions*.

Integration Testing

Ensuring that different modules or services *work together* correctly.

Critical for "*Build + Buy*" strategies. You must test if your custom software can successfully talk to "bought" APIs (e.g., Stripe, Supabase).

Verifying API connections, database queries, and data flow between internal and external systems.

Also runs through *assertions*.

Evaluation

Evaluation, in this context, ensures that all the functional requirements of the system are met.

Also done through *automated testing* and *user run throughs* though often with the aid of an SRS document.

Techniques for evaluation include Feature Testing, User Acceptance Testing, and Pilot Programming.

Feature Testing

Testing a complete "*feature*" or user flow from the *user's perspective*.

Used during **Evaluation** to confirm if a "Buy" solution actually delivers the *specific workflow* our business needs.

Does the "Add to Cart" or "Sign Up" button actually result in the expected outcome?

And other tests defined in your *functional requirements* and *use case diagrams*.

Pilot Programming

Deploying the system to a *limited* group of users or a single department before a full-scale launch.

Particularly useful for *Buy* decisions to test how a third-party product integrates with *existing company culture and workflows*.

Allows for identifying "*showstopper*" functional issues in a controlled environment with minimal impact.

User Acceptance Testing

The last phase where *end-users* test the system in a *real-world environment*.

The ultimate test of *Evaluation*. If users reject the system, the organization must decide whether to iterate the build or seek an alternative solution.

To ensure the system is "*fit for purpose*" and ready for production.

Benchmarking

Measuring the system against "How well it should do it" (Performance, Scalability, Reliability).

Used to *compare* a potential "Buy" solution against a current "Build" prototype or other "Buy" solutions.

Used to give hard numbers to things like:

- *Latency*: Time to first byte / Response speed.
- *Throughput*: Requests per second (Capacity).
- *Error Rate*: Percentage of failed requests under heavy load.

Benchmarking techniques include: Load Testing, Soak Testing, Spike Testing, Latency Benchmarking, and Availability Benchmarking.

Load Testing

Load Testing is a key technique used in benchmarking.

This is where the developers *artificially increase traffic* to see where the system slows down or crashes.

Essential for benchmarking a vendor's SLA (Service Level Agreement).

Does their cloud hosting actually handle 10,000 users as promised?

Primarily focuses on throughput and response time under pressure.

Soak Testing

Testing the system's stability over a *long period* of time (e.g., 24-48 hours) to find memory leaks or performance degradation.

Used to ensure a "Buy" solution is stable under continuous load and doesn't slow down over days of use.

Spike Testing

Testing the system's ability to handle *sudden, extreme bursts* of traffic.

Testing how a cloud provider (like AWS or Vercel) handles a sudden "viral" moment. Does it scale up fast enough, or does it crash?

Measuring recovery time after the traffic surge and ensuring the system doesn't permanently fail.

Latency Benchmarking (P99/P95)

Measuring the response time for the *slowest 1% or 5%* of users to ensure consistent performance.

If 95% of users get a fast response but 5% wait 10 seconds, the "Buy" solution might have architectural flaws.

These tests ensure a consistent user experience even under heavy load, rather than just looking at "average" speed.

Availability Benchmarking

Verifying the historical "*Uptime*" and reliability of a third-party vendor.

Comparing the "*Five Nines*" (99.999%) claims of a vendor against their actual historical performance records.

Validating if the vendor's infrastructure is reliable enough to support your business-critical systems.